LangGraph 기본 구성요소 (실습)

2-1. State 정의 (실습)
2-2. Node와 Edge (실습)
2-3. 그래프 컴파일과 실행 (실습)

2-1. State 정의 (실습)

> 그래프 전체가 공유하는 상태 스키마를 만들고 실행 시점의 정체를 확인해봅니다.

가장 단순한 State부터 만들어 봅니다. State 설계는 두 가지 질문에 답하는 일입니다. 그래프가 시작할 때 무엇을 받는지 정하면 입력 필드가 됩니다.
각 노드가 다음 노드에게 무엇을 남겨야 하는지 정하면 중간 결과와 최종 결과 필드가 됩니다. 이렇게 나온 답을 필드로 나열하면 State 스키마가 됩니다.

따라하기

1. 실습 레포 루트(`sds-ax-practice`)에서 오늘 실습 폴더를 만들고, 1일차에 만든 가상환경을 그대로 활성화합니다. 오늘 실습 파일은 모두 이 `day3_practice` 폴더 안에 저장합니다.

```
mkdir day3_practice
cd day3_practice
..\.venv\Scripts\activate
# 오늘 쓰는 langgraph는 1일차 requirements.txt에 들어 있습니다 (추가 설치 없음)
```

2. 자격 증명은 1일차에 레포 루트에 만들어 둔 `.env` 를 그대로 씁니다. 복사할 필요 없습니다.

3. 첫 State를 정의하고 실행 시점에 무엇인지 확인합니다. 아래 코드를 `day3_01_state.py` 로 저장합니다.

```
# day3_01_state.py - State 스키마 정의
from typing import TypedDict


class State(TypedDict):
    user_input: str   # 사용자가 넣는 입력
    response: str     # 그래프가 채울 응답


initial: State = {"user_input": "안녕하세요", "response": ""}
print(initial)
print(type(initial))   # 실행 시점에는 그냥 dict입니다
```

막힐 때

- `ImportError: cannot import name 'TypedDict'` 가 납니다.

  Python 3.10 이상인지 `python --version` 으로 확인합니다. 가상환경이 활성화되지 않은 채 시스템의 옛 파이썬으로 실행한 경우가 많습니다.

- 편집기가 langgraph를 못 찾는다고 표시합니다.

  편집기의 파이썬 인터프리터를 레포 루트의 `sds-ax-practice\.venv` 안의 파이썬으로 지정합니다.

확인: 출력에 딕셔너리 내용과 `<class 'dict'>` 가 나오면 성공입니다.

> State 스키마는 선언이고 실행 시점의 실체는 dict라는 걸 확인했습니다.

2-2. Node와 Edge (실습)

> 함수를 노드로 등록하고 시작과 끝을 엣지로 연결해봅니다.

노드는 그냥 파이썬 함수입니다. 규칙은 두 개뿐입니다. 인자로는 그래프의 State 하나만 받습니다. 반환은 State 전체가 아니라 바꿀 키만 담은 dict로 합니다. 이때 반환된 dict를 LangGraph가 기존 State에 병합(merge)합니다.

```
# LangGraph가 내부에서 하는 일 (개념적으로)
state = {"user_input": "안녕", "response": ""}
node_returned = {"response": "입력하신 내용은 '안녕' 입니다."}
new_state = {**state, **node_returned}   # 같은 키는 덮어쓰기가 기본
```

위 코드의 `{**state, **node_returned}` 는 dict 두 개를 펼쳐 하나로 합치는 문법입니다. 같은 키가 양쪽에 있으면 뒤에 온 쪽이 이깁니다.

노드 함수는 순수(pure)하게 작성해야 합니다. 들어온 State를 직접 수정하지 않고 갱신분만 새 dict로 반환합니다.

LangGraph의 핵심 기능이 전부 이 전제 위에서 돌아가기 때문입니다. 체크포인트 저장과 복원, 과거 시점으로 되돌리는 디버깅, 병렬 실행이 여기에 해당합니다.

따라하기

1. 노드 함수를 만들고 일반 함수처럼 직접 호출해 동작을 확인합니다. 노드가 결국 함수라는 것을 몸으로 확인하는 단계입니다. 아래 코드를 `day3_02_hello_graph.py` 로 저장합니다.

```
# day3_02_hello_graph.py (작성 시작)
from typing import TypedDict
from langgraph.graph import StateGraph, START, END


class State(TypedDict):
    user_input: str
    response: str


def respond(state: State) -> dict:
    user = state["user_input"]                              # 1) State에서 읽고
    return {"response": f"입력하신 내용은 '{user}' 입니다."}  # 2) 갱신분만 반환


# 노드는 그냥 함수이므로 그래프 없이도 직접 호출해 볼 수 있습니다
print(respond({"user_input": "테스트", "response": ""}))
```

2. 함수를 그래프에 등록하고 흐름을 연결합니다. 위 파일에 이어서 작성합니다.

```
builder = StateGraph(State)              # State 스키마를 알려주며 빌더 생성
builder.add_node("respond", respond)     # (노드 이름, 함수)

builder.add_edge(START, "respond")       # 시작에서 respond로
builder.add_edge("respond", END)         # respond에서 끝으로
```

`add_node` 의 첫 인자는 노드 이름(문자열)입니다. 이후 엣지를 연결할 때 이 이름을 씁니다. 이름은 retrieve와 generate처럼 동작을 나타내는 동사로 짓는 것이 좋고 step1과 node_a는 피합니다.

`START` 와 `END` 는 LangGraph가 제공하는 특수 노드입니다. 그래프는 분기와 순환을 지원하므로 어디서 시작해서 어디서 끝나는지 반드시 명시해야 합니다.

막힐 때

- 노드가 dict가 아니라 값을 그대로 반환하면 이후 실행에서 에러가 납니다.

  `return response`가 아니라 `return {"response": response}`로 씁니다. LangGraph는 반환값의 키를 갱신할 필드로 해석합니다.

- 노드 이름과 함수 이름이 달라도 되는지 헷갈립니다.

  달라도 됩니다. 다만 같게 두는 것이 읽기 편합니다.

확인: 1단계 직접 호출에서 `{'response': "입력하신 내용은 '테스트' 입니다."}`가 출력되면 성공입니다.

> 노드는 State를 받아 갱신분 dict만 반환하는 평범한 함수라는 걸 확인했습니다.

2-3. 그래프 컴파일과 실행 (실습)

> 첫 그래프를 컴파일해 실행하고 다이어그램 시각화로 구조를 확인해봅니다.

빌더는 설계도일 뿐이라 바로 실행할 수 없습니다. `compile()`을 거쳐야 실행 가능한 객체가 됩니다. 이때 LangGraph가 구조 검증(연결 안 된 노드가 있는지 등)도 함께 합니다.

따라하기

1. 2-2에서 만든 `day3_02_hello_graph.py`를 아래의 완성된 전체 코드로 채우고 실행합니다.

```
# day3_02_hello_graph.py - 첫 그래프 전체 코드
from typing import TypedDict
from langgraph.graph import StateGraph, START, END


class State(TypedDict):
    user_input: str
    response: str


def respond(state: State) -> dict:
    user = state["user_input"]
    return {"response": f"입력하신 내용은 '{user}' 입니다."}


builder = StateGraph(State)
builder.add_node("respond", respond)
builder.add_edge(START, "respond")
builder.add_edge("respond", END)

graph = builder.compile()    # 그래프를 실행 가능한 객체로 변환

result = graph.invoke({"user_input": "안녕하세요"})
print(result)

# 구조 시각화: PNG 저장이 안 되는 환경도 있으므로 try/except로 감쌉니다
try:
    graph.get_graph().draw_mermaid_png(output_file_path="hello_graph.png")
    print("hello_graph.png 저장 완료")
except Exception as e:
    print("PNG 저장은 건너뜁니다:", e)
    print(graph.get_graph().draw_mermaid())   # 텍스트 출력으로 대체
```

2. 출력에서 세 가지를 확인합니다.
   - `graph.invoke(...)`는 1일차와 2일차에서 쓴 체인의 invoke와 같은 인터페이스입니다. 컴파일된 그래프도 LangChain Runnable이기 때문입니다.
   - 반환값은 최종 State 전체입니다. 입력으로 넣은 `user_input`과 노드가 채운 `response`가 함께 들어 있습니다.
   - invoke에 `response` 키를 빼고 넣어도 동작합니다. 노드가 채우기 전까지 그 키가 없을 뿐입니다.

막힐 때

- `builder.invoke(...)` 를 호출했더니 에러가 납니다.

  빌더에는 invoke가 없습니다. `compile()` 을 거친 객체만 실행할 수 있습니다.

- `draw_mermaid_png()` 에서 에러가 납니다.

  환경에 따라 그림 렌더링에 추가 설치가 필요합니다. 시간을 쓰지 말고 위 코드의 except 경로처럼 `draw_mermaid()` 텍스트 출력으로 대체합니다. 출력된 텍스트를 mermaid.live 같은 사이트에 붙여 넣으면 그림으로 볼 수 있습니다. 시각화는 학습 보조 수단이지 필수가 아닙니다.

- 컴파일할 때 dead-end 노드 에러가 납니다.

  `add_edge("respond", END)` 를 빼먹은 경우입니다. 모든 노드는 어딘가로 나가는 길이 있어야 합니다.

확인: `{'user_input': '안녕하세요', 'response': "입력하신 내용은 '안녕하세요' 입니다."}` 가 출력되면 성공입니다.

> State 정의, 노드 등록, 엣지 연결, 컴파일, 실행이라는 그래프 개발의 기본 사이클을 익혔습니다.
